

FULL STACK

ASSIGNMENT – 1

IMPLEMENT INHERITANCE TYPES USING JAVASCRIPT

Student Name: D. Vijaya Sri

Roll Number: A24126552262

Department: CSM (AI&ML)

Section: B

Subject: FULL STACK

Assignment No.: 1

Date: 29-08-2026

Academic Year: 2026

Technology Used: JavaScript with Node.js

1. Objective

To implement and demonstrate different types of inheritance in JavaScript using Node.js, and understand how classes and objects can reuse properties and methods.

2. Introduction

Inheritance is an object-oriented programming concept in which a child class can acquire methods and behavior from a parent class. JavaScript supports class-based inheritance through the **extends** keyword. Multiple inheritance is not directly supported by JavaScript classes, so mixins can be used to combine behavior from more than one source.

3. Types of Inheritance Implemented

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance using Mixins
- Hybrid Inheritance

4. System / Software Requirements

- Node.js
- Visual Studio Code or any text editor
- Windows / Linux / macOS
- Command-line terminal

1. Single Inheritance

Definition: A child class inherits from a single parent class.

Source Code:

```
// Single Inheritance

class Animal {
  eat() {
    console.log("Animal can eat.");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog can bark.");
  }
}

const dog = new Dog();

dog.eat();
dog.bark();
```

Execution: Run the file from the assignment source folder using Node.js.

2. Multilevel Inheritance

Definition: A class inherits from a class that already inherits from another class, forming a chain.

Source Code:

```
// Multilevel Inheritance

class Animal {
  eat() {
    console.log("Animal can eat.");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog can bark.");
  }
}

class Puppy extends Dog {
  play() {
    console.log("Puppy can play.");
  }
}

const puppy = new Puppy();

puppy.eat();
puppy.bark();
puppy.play();
```

Execution: Run the file from the assignment source folder using Node.js.

3. Hierarchical Inheritance

Definition: Multiple child classes inherit from the same parent class.

Source Code:

```
// Hierarchical Inheritance

class Animal {
  eat() {
    console.log("Animal can eat.");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog can bark.");
  }
}

class Cat extends Animal {
  meow() {
    console.log("Cat can meow.");
  }
}

const dog = new Dog();
const cat = new Cat();

dog.eat();
dog.bark();

cat.eat();
cat.meow();
```

Execution: Run the file from the assignment source folder using Node.js.

4. Multiple Inheritance (using Mixins)

Definition: JavaScript does not support direct multiple class inheritance; mixins are used to combine behavior from multiple classes.

Source Code:

```
// Multiple Inheritance using Mixins

class Father {
  fatherSkill() {
    console.log("Father: Driving");
  }
}

class Mother {
  motherSkill() {
    console.log("Mother: Cooking");
  }
}

class Child extends Father {
  childSkill() {
    console.log("Child: Coding");
  }
}

// Copy Mother's methods into Child
Object.getOwnPropertyNames(Mother.prototype).forEach(method => {
  if (method !== "constructor") {
    Child.prototype[method] = Mother.prototype[method];
  }
});

const child = new Child();

child.fatherSkill();
child.motherSkill();
child.childSkill();
```

Execution: Run the file from the assignment source folder using Node.js.

5. Hybrid Inheritance

Definition: A combination of different inheritance approaches, demonstrated here with class inheritance and a mixin.

Source Code:

```
// Hybrid Inheritance

class Animal {
  eat() {
    console.log("Animal can eat.");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog can bark.");
  }
}

class Cat extends Animal {
  meow() {
    console.log("Cat can meow.");
  }
}

class Pet extends Dog {
  play() {
    console.log("Pet can play.");
  }
}

// Mixin
const Friendly = {
  beFriendly() {
    console.log("Pet is friendly.");
  }
};

Object.assign(Pet.prototype, Friendly);

const pet = new Pet();

pet.eat();
pet.bark();
pet.play();
pet.beFriendly();
```

Execution: Run the file from the assignment source folder using Node.js.

6. Output

The following screenshot shows the successful Node.js execution output for all five inheritance programs.

```
c:\Users\dadhi\FULL_STACK\03-assignments\assignment-1\src-Inheritance-types-using-javascript>node singleInheritance.js
Animal can eat.
Dog can bark.

c:\Users\dadhi\FULL_STACK\03-assignments\assignment-1\src-Inheritance-types-using-javascript>node multilevelInheritance.js
Animal can eat.
Dog can bark.
Puppy can play.

c:\Users\dadhi\FULL_STACK\03-assignments\assignment-1\src-Inheritance-types-using-javascript>node hierarchicalInheritance.js
Animal can eat.
Dog can bark.
Animal can eat.
Cat can meow.

c:\Users\dadhi\FULL_STACK\03-assignments\assignment-1\src-Inheritance-types-using-javascript>node multipleInheritance.js
Father: Driving
Mother: Cooking
Child: Coding

c:\Users\dadhi\FULL_STACK\03-assignments\assignment-1\src-Inheritance-types-using-javascript>node hybridInheritance.js
Animal can eat.
Dog can bark.
Pet can play.
Pet is friendly.
```

Result: All five inheritance implementations were executed successfully using Node.js.

Conclusion: The assignment demonstrates how inheritance can be implemented in JavaScript using classes, the extends keyword, and mixins for combining behaviors.